in the *Projects/www* subdirectory of my home directory, I need to enter the URL *http://localhost/~saurav/test.php*, where *saurav* is my username. Similarly, if *test.php* is in a subdirectory of *Projects/www* (for example, *Projects/www/test/test.php*), I need to enter *http://localhost/~saurav/test/test.php*.

Warning: If you have a system with extra security such as SELinux or AppArmor enabled, and it is configured to prevent programs like Web servers from accessing your files (such as the default configuration on Fedora), then Lighttpd may not be able to serve your pages. The recommended method to correct this is to set the correct policies in your security system to selectively allow access. The easier (and riskier!) method is to change the security system's settings to merely warn about access, instead of preventing it.

Apart from this, you need to ensure that the user account under which Lighttpd runs (usually named *lighttpd*) has read access to the directory from which you want to serve Web pages, and read-and-write access to your SQLite databases and the directory containing them.

SQLite

Relational database management systems, being based on a clean and well-implemented design founded on mathematical principles, offer a lot of convenience for managing data in a standard manner, while allowing ad-hoc queries on the database. Most relational database management systems are implemented as database servers, which require the user (administrator) to monitor and tune the server for performance. A server is required when multiple simultaneous client connections need to be served—but that doesn't mean that relational databases can only be used with database servers. You may prefer a server-less system in many situations—for example, using a database like an ordinary file for a single user or application, especially on a system with low resources. Being able to use a relational database and run SQL queries on it in such situations can eliminate a lot of file-handling and information-parsing headache, allowing you to make use of a medium that you are already familiar with. SQLite is one tool that enables this.

Warning: Never use SQLite in a situation where multiple connections to the database will be maintained simultaneously. There are no guarantees that the system will work properly. Serving multiple clients simultaneously is the job of a database server, and is best left to it. SQLite is not a server. You can, of course, use separate database files with separate connections to emulate a server environment—but the details of that are up to you, and it does not scale to handle large loads like a proper server.

The homepage of SQLite is at *http://www.sqlite.org/*. SQLite is a tool that stores a relational database in a single file. You can create, populate and query database tables using SQL within that file. You can copy the file and use it anywhere SQLite is installed. This article demonstrates how to install and use SQLite on Fedora; the procedure is similar for other systems.

Installing SQLite and allied packages

There are two versions of *SQLite* currently in use, versions 2 and 3, one or both of which are sometimes already installed on GNU/Linux systems. We will use version 3 here, because it is the latest one. On Fedora, using your package-management system, install the sqlite package if it is not already installed. The *sqlite2* package provides SQLite version 2, and you can install both side by side. You may also want to install the *sqlite-doc* package to get offline access to the SQLite documentation.

To use SQLite 3 with PHP, you need to access it through PDO. Install PDO if it is not already installed; the Fedora package is *php-pdo*. You can check whether the SQLite PDO driver is available by running the PHP statement *print_r(PDO::getAvailableDrivers())*;. This is all that is required to use SQLite.

SQLite is very simple to use through the text-mode program provided with it, but if you prefer a graphical interface, install the *sqliteman* package. We will use the text-mode program for the purposes of this article.

The quirks of SQLite

Though SQLite provides an SQL interface to manage data, it is one of the quirkiest database management systems. It mostly adheres to SQL92, but there are some areas where it deviates, and there are a few features it does not provide. For example, the *ALTER TABLE* statement implemented in it provides a facility to add and rename columns, but not to drop or alter them. Also, there is no concept of *GRANT* and *REVOKE* because the database is stored in a file, and is intended to be used by a single process at a time—so only the file-system privileges for the database file govern its use.

The part of SQLite that differs the most from other systems is its handling of data types. SQLite does not associate data types with columns or fields. Rather, it associates the type with the data values, similar to dynamically typed programming languages. One of the consequences of this is that you can store any type of data in any type of field. Column types are specified in the *CREATE TABLE* statement just as in other, more traditional systems, but the type only serves as a hint to the type of data expected in the field, rather than as a constraint. When you try to enter a value in a field, SQLite tries to convert it to the field's specified type—but even if it cannot perform the conversion, it will still store the value.